

Helix VPN — Phase 2 (MVP2) Client Applications

Helix VPN — Phase 2 (MVP2) Client Applications

Comprehensive Technical Specification Package

Revision: 2 **Last modified:** 2026-07-04T12:00:00Z

Version: 1.1 **Date:** 2026-07-03 (Revision 2: 2026-07-04) **Status:** Final Draft **Classification:** Internal — Development Team

Revision 2 changelog: deep gap-analysis + production-readiness hardening pass across the full MVP2 corpus — reconciled cross-document contradictions (timeline, minimum OS versions, protocol support claims) and added Enterprise Hardening / Production Readiness sections plus Mermaid architecture diagrams to every platform document. See MVP2_OVERVIEW.md §7.8 for the consolidated summary of what changed.

Executive Summary

This package contains the complete technical specification for **Helix VPN Phase 2 (MVP2)** — the client application layer of the Helix VPN ecosystem. MVP2 covers all client-facing applications across **8 platforms** and **7 operating systems**, built on a unified Rust shared core architecture for maximal code reusability (70-85%), minimal footprint, and native performance.

Target Platforms

Platform	OS	UI Framework	Bundle Target	Code Reuse
Desktop	macOS	Tauri v2	< 15 MB	85%
Desktop	Windows	Tauri v2	< 15 MB	80%
Desktop	Linux	Tauri v2	< 15 MB	85%
Mobile	Android	Flutter	< 25 MB	72%
Mobile	iOS	Flutter	< 25 MB	72%
Mobile	HarmonyOS	Flutter (ohos)	< 25 MB	70%
Desktop	Aurora OS	Qt6/QML	< 20 MB	75%
Web	Browser	Extension + Tauri	< 5 MB	45%

Key Metrics

- **Total Documentation:** 24,000+ lines across 10 specification documents
- **Research Base:** 8 wide-exploration research briefs (8,300+ lines)
- **Estimated Implementation:** 36 weeks (expected case; 30 weeks best case / 44 weeks worst case — see MVP2_IMPLEMENTATION_ROADMAP.md §13) with an 11-person team
- **Code Reuse Target:** 70-85% across all platforms via Rust shared core
- **Protocols Supported:** WireGuard (primary), Shadowsocks SIP022 AEAD-2022 (secondary), MASQUE/QUIC RFC 9298 (secondary), Multi-Hop (advanced). **OpenVPN is a reserved ProtocolType enum placeholder only** (openvpn = [] empty Cargo feature flag) — not implemented or supported in MVP2; see MVP2_SHARED_CORE.md §2.3 and corrected in this revision (earlier drafts listed it as supported).

Documentation Index

Core Documents (Read in Order)

#	Document	File	Lines	Description
1	Overview	MVP2_OVERVIEW.md	696	Executive summary, goals, platform matrix,

#	Document	File	Lines	Description
				success criteria
2	Architecture	MVP2_ARCHITECTURE.md	1,257	System architecture, component diagrams, technology stack decisions
3	Shared Core	MVP2_SHARED_CORE.md	2,482	Rust core library specification, FFI design, cross-compilation
4	Desktop Apps	MVP2_DESKTOP_APPS.md	3,225	macOS, Windows, Linux client specifications with code examples
5	Mobile Apps	MVP2_MOBILE_APPS.md	4,924	Android, iOS, HarmonyOS client specifications
6	Web Client	MVP2_WEB_CLIENT.md	2,785	Browser extension, admin panel, PWA specification
7	Aurora Client	MVP2_AURORA_CLIENT.md	2,564	Aurora OS Qt/QML client with Silica UI
8	Security & Performance	MVP2_SECURITY_PERFORMANCE.md	2,304	Security architecture, kill switch, CI/CD pipeline
9	UI/UX Design	MVP2_UI_UX_SPEC.md	1,456	Design system, components, screen specifications
10	Implementation Roadmap	MVP2_IMPLEMENTATION_ROADMAP.md	2,553	36-week phased plan

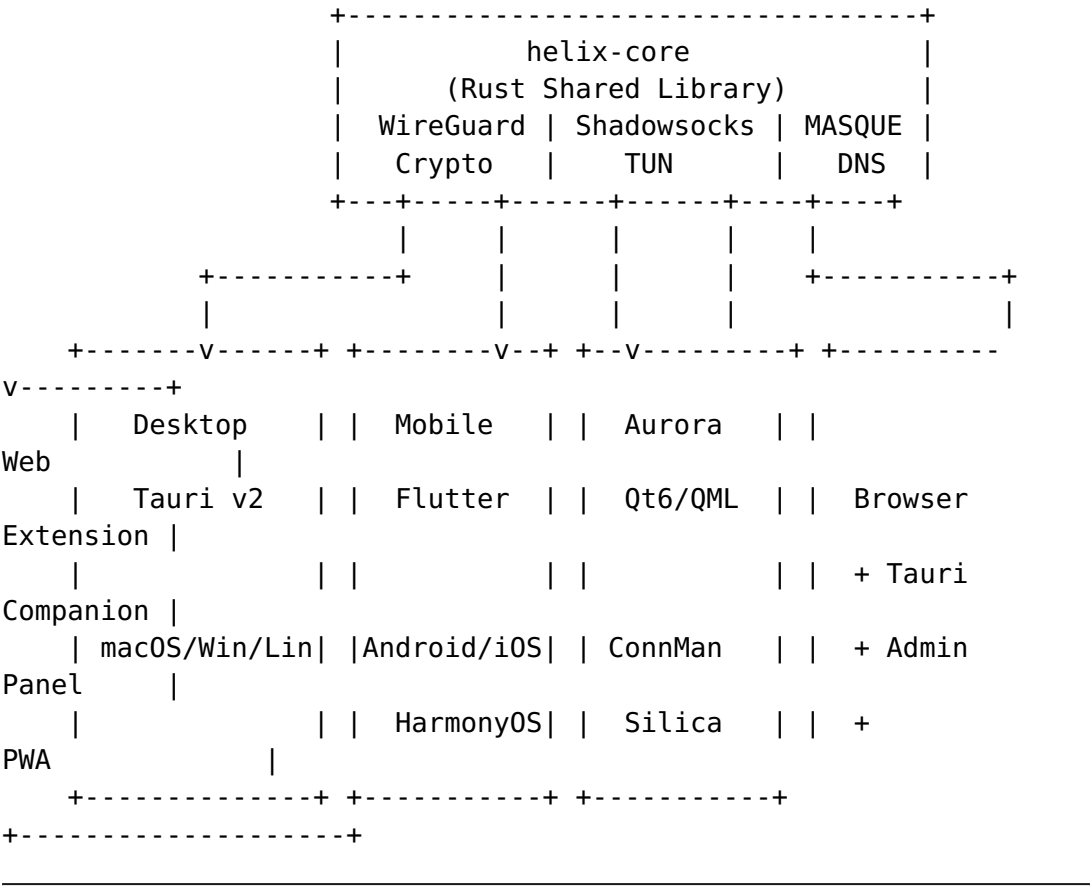
#	Document	File	Lines	Description
				with milestones

Research Documents

Located in the research/ directory, containing the raw research that informed all specifications:

Document	Description
mvp2_wide01.md	Cross-platform UI frameworks comparison
mvp2_wide02.md	OS-specific VPN integration APIs
mvp2_wide03.md	VPN protocol implementations in Rust
mvp2_wide04.md	Mobile platform implementation details
mvp2_wide05.md	Desktop platform implementation details
mvp2_wide06.md	Web platform capabilities
mvp2_wide07.md	Shared core architecture patterns
mvp2_wide08.md	Security, performance, and build pipeline

Architecture Overview



Quick Start for Stakeholders

For Engineering Leads

Read in order: Overview → Architecture → Shared Core → Implementation Roadmap

For Platform Developers

Read: Overview → Architecture → [Your Platform] → Shared Core → Security & Performance

For UI/UX Designers

Read: Overview → UI/UX Design → [Platform Screens] → Architecture (UI sections)

For Project Managers

Read: Overview → Implementation Roadmap → Architecture (decisions only)

For Security Reviewers

Read: Overview → Security & Performance → Shared Core (security section)

Design Principles

1. **Rust-First Core** — All business logic lives in a memory-safe, high-performance Rust shared library
 2. **Maximal Reuse** — 70-85% code reuse across platforms via shared core and unified UI frameworks
 3. **Native Performance** — Each platform uses native APIs (NetworkExtension, VpnService, WFP, TUN)
 4. **Minimal Footprint** — Desktop < 15MB, Mobile < 25MB, Web < 5MB bundle targets
 5. **Security by Design** — Kill switch, leak prevention, post-quantum crypto, secure key storage
 6. **Platform Native** — UI follows each platform's design language while maintaining brand consistency
-

Implementation Timeline

Phase	Duration	Deliverables
Phase 1: Foundation	Weeks 1-4	Rust core, CI/CD, basic TUN
Phase 2: Protocols	Weeks 3-8	WireGuard, Shadowsocks, DNS
Phase 3: Desktop macOS/Linux	Weeks 6-14	Tauri apps for macOS and Linux
Phase 4: Desktop Windows	Weeks 12-18	Windows Tauri app
Phase 5: Mobile Android	Weeks 14-22	Flutter Android app
Phase 6: Mobile iOS	Weeks 20-26	Flutter iOS app
Phase 7: HarmonyOS/Aurora	Weeks 24-30	Extended platform clients
Phase 8: Web	Weeks 28-34	Browser extension + admin
Phase 9: Launch	Weeks 32-36	Testing, audit, production

Total Expected Duration: 36 weeks (9 months)

Team Size: 11 engineers

Parallel Workstreams: Up to 3 during peak phases

Technology Stack Summary

Layer	Technology	Purpose
Core	Rust + Tokio	VPN engine, protocols, crypto
Desktop UI	Tauri v2 + React/Vue	macOS, Windows, Linux apps
Mobile UI	Flutter + Dart	Android, iOS, HarmonyOS apps
Aurora UI	Qt6 + QML + Silica	Aurora OS app
Web UI	React + TypeScript	Browser extension, admin panel
FFI	UniFFI + flutter_rust_bridge	Language bindings
Protocols	boringtun + shadowsocks-rust	WireGuard + Shadowsocks

Layer	Technology	Purpose
Crypto	rustls + x25519-dalek	TLS, key exchange
Build	GitHub Actions + cargo-cross	CI/CD, cross-compilation

Contact & Contribution

- **Project Repository:** `git@github.com:HelixDevelopment/helix_vpn.git`
- **Documentation Updates:** Submit PRs against `docs/research/mvp2/` directory
- **Issue Tracking:** Use GitHub Issues with `mvp2` label

This documentation package was generated on 2026-07-03 as part of the Helix VPN MVP2 planning phase, and hardened for production-readiness on 2026-07-04 (Revision 2 — deep gap analysis, contradiction reconciliation, Enterprise Hardening sections, and Mermaid architecture diagrams across all ten specification documents). All specifications are final draft and subject to engineering review before implementation begins.